Uscha: A Fact-Gated, Tool-Agnostic Methodology for Specification-Driven Development with LLM Coding Agents

Andrés Massello*
Independent Researcher — Córdoba, Argentina
info@uscha.dev · github.com/andresmassello · linkedin.com/in/amassello
DOI: https://doi.org/10.5281/zenodo.21228014

July 2026 (revised September 2026)

Abstract

LLM coding agents fail in two well-documented ways: when instructions are underspecified they guess instead of stopping, and when a verification signal is visible they optimize the signal rather than the requested outcome. We present **Uscha**, a methodology for specification-driven development with LLM coding agents organized around a single discipline: *gates that read facts block; gates that guess over prose advise.* The methodology contributes (i) two specification fronts — a greenfield front where the agent *proposes* the system shape under one-question-at-a-time human approval, and a brownfield front where the agent extracts verifiable facts and may author only *candidate* claims in quarantine, promoted to the contract by human verdict alone; (ii) a deterministic evidence ledger with a two-tier record contract in which measured artifacts always override agent self-reports; (iii) a family of executable fact gates (golden non-authorship, gate-integrity, simplicity budgets, per-criterion measured acceptance, regression capture, a derived — never self-declared — workflow state machine) and a structured-guess layer (versioned qualitative rubrics with mandatory evidence, and deterministic clone-vs-repository detection that surfaces the duplication a diff-local simplicity gate cannot see) that advises by default and gates only by explicit human declaration; and (iv) a strict placement of the human at three fixed points: approving the shape, approving the golden baseline, and merging; and (v) a closed forward/reverse cycle — the specification package with its behavior ledger and intermediate representation as the canonical asset of a system, code as a regenerable build artifact, an LLM as the compiler under a validated output contract and reverse discovery as the decompiler — stated as a thesis and put under its own instruments rather than asserted. The methodology also applies its own Lean discipline reflexively: a meta-invariant that keeps a growing set of good gates from becoming an audit (over-processing waste), enforced by collapsing every persisted gate into a single readiness verdict. A reference implementation (nine agent skills and a dependency-free Python engine with 63 subcommands and adapters for eleven language stacks) is described, together with its design validation: two adversarial audits (231 agent evaluations) that initially found the central principle inverted in code and drove its re-wiring; a systematic cross-examination against classical software-engineering doctrine that produced ten shipped improvements; and sustained self-application, in which fresh-context reviews repeatedly caught real defects before merge across the release series. We report design validation, self-application, and one bounded controlled evaluation — the Diamond program, a blind, oracle-certified benchmark of implementation replaceability across twelve archetypes, compiled by four models across two vendors and qualified by a measured noise floor; a controlled evaluation on external operators remains future work.

*This work is licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0) license.

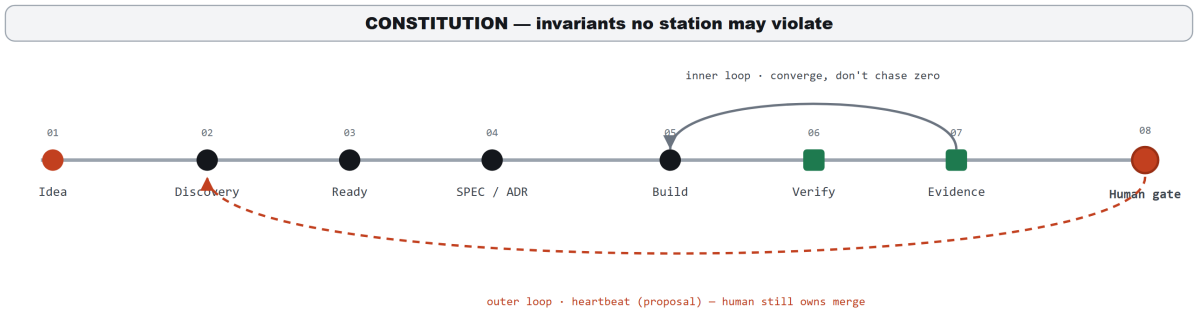


Figure 1: The Uscha pipeline. Eight stations from idea to human gate under a constitution of inviolable invariants. The inner loop (build–verify–evidence) converges instead of chasing zero findings; the outer loop returns evidence to discovery as a proposal — the human still owns the merge.

1 Introduction

Two recent empirical results frame the problem this methodology addresses. First, when task instructions are underspecified, production coding agents do not fail closed: they act on unstated assumptions, and a majority of runs violate at least one action boundary [1]. Second, when a verification oracle is visible to the agent, near-perfect scores can coexist with an undelivered artifact: the agent *builds to the test*, optimizing the check rather than the request, and does not on its own validate what it ships as a user would [2]. Both failures are dispositional rather than capability failures, and both worsen as agents are given more autonomy.

Existing agent development frameworks converge on persistent artifacts and human review as mitigations [3], but they largely encode their process as *prose instructions that the agent may ignore*, and their verification signals rarely distinguish between what is mechanically verifiable and what is a model's opinion. Uscha[1] starts from that distinction and makes it the organizing principle of the whole process. Figure 1 shows the resulting pipeline: eight stations from idea to human gate, under a constitution of invariants that no station may violate, with an inner convergence loop over the build and an outer proposal loop that returns to discovery — while the merge remains human.

A useful way to see the resulting shape is as a choice of control flow. The default agentic pattern is an *open loop* — investigate, edit, test, and then let the agent answer "is it resolved?" itself — and that self-assessment is exactly the signal the two results above show to be unreliable. Uscha instead runs the same work through a *measured graph*: the steps become defined stations whose transitions are gated by computational evidence in the ledger rather than by the agent's own verdict. The loop is not removed but bounded — it lives inside one station (build–verify convergence) that either converges or escalates to a human when it stalls, instead of iterating on the model's judgment of its own doneness. What the practitioner authors is not the topology, which is fixed, but the guardrails the graph may not cross: the specification, the acceptance oracle, and the constitution of invariants.

The methodology is built on one discipline and three architectural commitments:

Facts block; guesses advise. Every check in the process is classified as *computational* (it reads an artifact — a byte comparison, a parsed test report, a diff structure) or *inferential* (it judges prose or code quality). Computational checks are allowed to block progress

[1] "Uscha" is the name of the methodology; the pattern it operationalizes is a *spec-loop* — a specification-driven build-and-verify loop. The name was chosen partly to avoid collision with *spec-loop* [18], an independently developed, actively maintained open-source project that shares the term and the spec-driven, review-per-increment philosophy while taking a different implementation path (shell-based reusable agent skills, task-local specifications); it was found late in this work's development, and the two are convergent rather than derivative.

mechanically. Inferential checks may only advise, unless a human explicitly declares them gating. A natural-language heuristic that blocks produces false positives, gets disabled, and dies; keeping guesses advisory is what keeps them alive.

Measured beats narrated. The evidence ledger records two tiers: records that are *measured* (parsed from a real artifact — a test report, a coverage file, linter output) and records that are *narrated* (an agent's own account of what it did, believed only because it says so). A measured red always overrides a narrated green. Acceptance criteria close per criterion only when a green, name-tagged test case exists in a report born of a recorded, still-current `test-run` (otherwise the criterion reads UNMEASURED) — a checked checkbox without a test is recorded as *narrated-only* and does not close.

The human at three fixed points. The human approves the *shape* (specification front), approves the *golden baseline* (the one artifact the agent is mechanically forbidden from authoring), and *merges*. The agent proposes, measures, and stops.

Tool-agnosticism by contract. The engine is dependency-free Python that never runs an LLM: it validates structures and ingests reports, and for tests it runs only the command the project declared (`test-run`). Wherever a model's judgment participates (e.g. rubric grading), the interface is a JSON contract; who produced the JSON — one vendor's agent, another's, or a human — is irrelevant to the ledger.

The rest of the paper places the approach against related work (§2), then describes the methodology (§3), its evidence engine (§4), the reference implementation (§5), and the design-validation evidence accumulated so far (§6), followed by limitations (§7) and future work (§8).

2 Related Work

Underspecification. Ji et al. [1] show that underspecified operational instructions cause agents to guess rather than stop, violating action boundaries in 55.8–67.8% of runs. Uscha's greenfield front is a direct response: the specification is produced through a structured interrogation in which the agent proposes and the human decides, and inviolable constraints are captured in a constitution file whose breach is a first-class blocker, never a trade-off.

Building to the test. Ma et al. [2] demonstrate that agents deliver what is checked, not what is requested, and name the missing disposition *validation self-awareness*. Uscha treats this as its central adversary: acceptance closes per criterion on measured test evidence (from a recorded, still-current `test-run`, else UNMEASURED); findings cannot be closed without new test material (*find bugs once*); qualitative verdicts require file-and-line evidence or they do not count; and the workflow state that authorizes a pull request is derived from ledger facts, never self-declared by the agent.

Process taxonomies. de Macedo [3] surveys agent development frameworks along six dimensions (specification, context, roles, execution, validation, portability) and identifies recurring risks: specification drift, artifact fragility, and platform dependence. Uscha addresses each explicitly: drift through a rebuild test that scores whether the specification package alone can regenerate the system; fragility through an integrity-checksummed ledger; and platform dependence through the contract architecture and a dependency-free engine.

Field-level evidence and the autonomy trajectory. A systematic review of agentic AI across the software development life cycle [16] finds agentic support concentrated downstream — implementation, testing, maintenance — while the upstream phases (requirements, specification, design) remain the least mature, and it identifies output verifiability as the primary enabler of adoption. Uscha is a direct wager on that gap: it invests its heaviest machinery precisely upstream, where that review reports agentic support to be thinnest, and makes verifiability — not autonomy — the axis it optimizes. The framing is deliberately narrow: the

review establishes that the upstream, verifiability-first niche is real and underserved; it does not, and cannot, measure Uscha itself, whose own evidence remains design validation and self-application (§6). The complementary direction is the autonomy-first multi-agent framework, of which ALMAS [17] is representative: specialized agents aligned with agile roles drive the loop toward less human involvement. Uscha takes the opposite stance on the same problem — the human stays at three fixed decision points (shape, golden, merge), and the reviewer's verdict is a recorded ledger fact backed by file-and-line evidence, never one more agent's self-report.

Practitioner doctrine. The methodology's check classification adapts the computational-versus-inferential distinction from Böckeler's *sensor* framework for coding agents [6, 11] — the axis on which "facts block, guesses advise" rests. Its gate-integrity stance — that a change must not weaken the apparatus that measures it, and that a high-blast-radius change needs a checker uncorrelated with the maker — follows the agentic-code-review red flags catalogued by Osmani [7]. Architecture decision records follow Nygard [5], extended with machine-checkable implementation plans. Golden-master capture follows approval-testing practice [12], with one addition central to the method: the agent may author the capture harness but is mechanically forbidden — by a pre-tool-use hook, not by instruction — from writing an approved fixture. The simplicity gate operationalizes the "Reduce" law of Maeda [8]. The methodology was also systematically cross-examined against classical doctrine [4]; §6 reports the ten resulting improvements. Two agent-native failure modes complete the lineage: the premature "done" token — Huntley's *Ralph Wiggum loop* [9] — is exactly what the derived state machine and objective gates refuse to accept, and the maker-checker separation the review loop enforces is the evaluator-optimizer pattern [10]. The design stance throughout is an application of Goodhart's law [13] to agent verification: any signal the agent can see, it will eventually optimize. Independent evidence for the premise behind `check-terminado` arrived from a different scale: Cursor's report on a week-long, thousands-of-agents build with no human in the loop [19] states that its first harness failed because the model proclaimed success while far from it, and that output quality tracked the quality of the specification rather than the infrastructure. The same report removed its independent judge as a bottleneck at that scale, which this paper does not attempt to reconcile: the single-agent setting studied here has the gate as its only correction mechanism, and a scale criterion for when a gate stops paying for itself is stated as an open question, not a result. Independent evidence at the review stage, rather than the build stage, comes from Garg [20]: a controlled study (accepted at GAISS 2026) found that a specification baseline did not raise reviewers' bug-detection rate over code-only review — the two were statistically indistinguishable — but raised the share of findings attributable to a named, approved requirement from zero to a majority. A reasoning-first baseline with no specification captured nearly all of the apparent gain on simpler tasks, and specification-anchored review cost more reviewer time. This is a different measurement than the Diamond program's replaceability claim — reviewer attribution and time, not regeneration under a withheld oracle — and is not presented here as validation of it; the confound the source itself names, that part of the effect attributed to "specifying first" may be a reasoning effect rather than the specification, is stated as the source states it.

Industry work on running many agents at once builds the agent "factory": orchestration, continuous integration, tracing, and access control. Cursor's report [19] is one instance, and Cloudflare's proposed Agent Development Lifecycle, as reported by Pop [21], is another, built from a workflow-orchestration layer, a CI layer, OpenTelemetry-compatible tracing of model calls, tool executions, and token use, and short-lived, task-bound credentials. The tracing layer — a durable, replayable record of what the agent did — is the same move as this paper's evidence ledger. What that line of work does not report is a measurement of whether the output is correct: the ADLC article itself gives no defect, coverage, or review-reduction data and describes its tools as early iterations. The contribution here is orthogonal to the factory: a measured quality contract for what a factory produces. The ADLC article is cited for positioning,

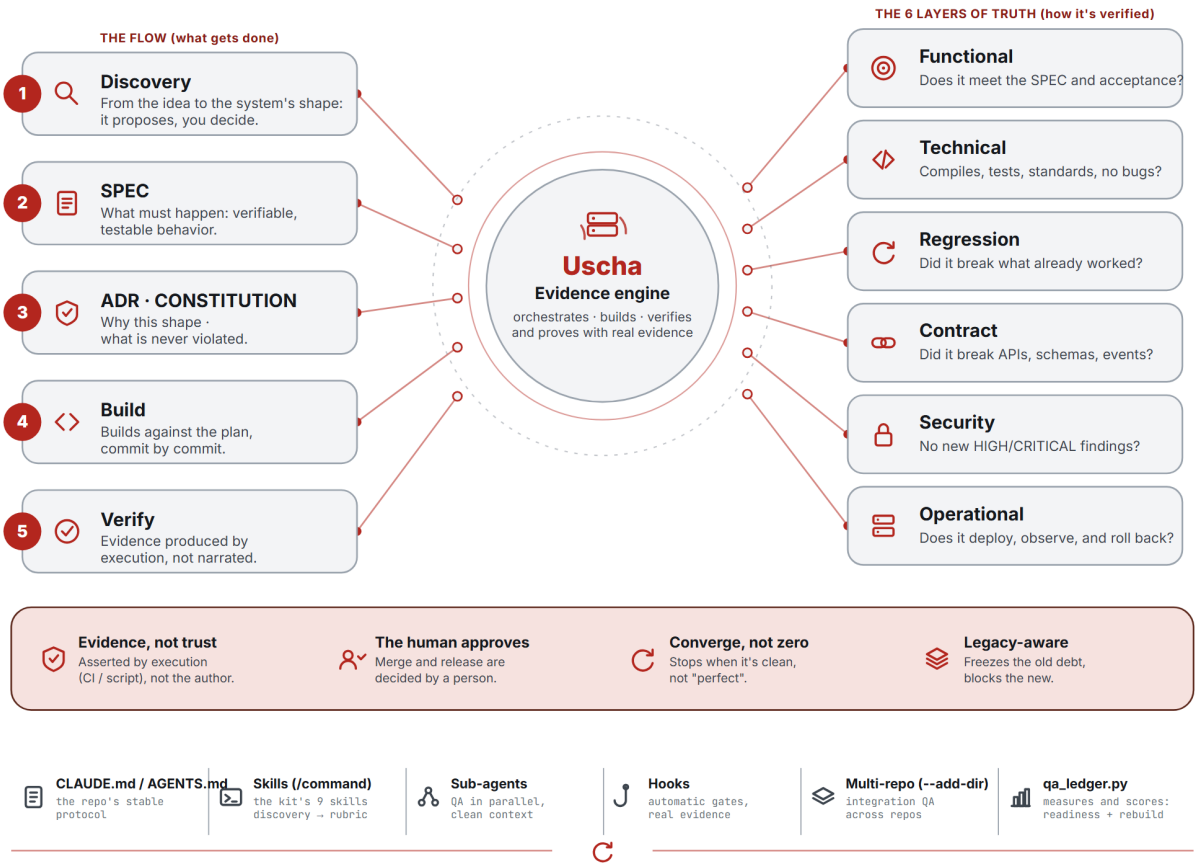


Figure 2: The five steps of the flow (left: what gets done) and the six layers of truth (right: how each result is verified), joined by the evidence engine. The band beneath states the four commitments the diagram encodes: evidence produced by execution rather than asserted by the author; merge and release decided by a person; convergence instead of perfection; and legacy awareness — freeze the old debt, block the new. The strip at the foot names the runtime surfaces the implementation binds to.

not as validation of any measurement in this paper.

3 The Methodology

3.1 Two specification fronts

Greenfield (discovery). The human brings an idea; the agent brings the *shape*. The front proceeds one question at a time, each question carrying the agent's recommended answer, walking a fixed agenda: purpose, domain model, operation surface, architecture options with trade-offs, dirty cases and failure behavior, inviolable constraints, out-of-scope, acceptance criteria with stable traceable identifiers, a declared quality bar, and residual risks. High-uncertainty risks trigger a time-boxed *spike* whose only legitimate output is a decision record with lessons — spike branches are mechanically refused at the pull-request gate. The front emits a versioned specification package: context glossary, constitution, domain model, specification, decision records with implementation plans and verification checklists, acceptance file, risks, and handoff. Figure 2 shows how the package's artifacts feed the evidence engine and how the engine verifies the result across six layers of truth.

Brownfield (reverse discovery). For an existing system, the direction of trust inverts: observable behavior is ground truth, and the agent begins by producing *facts* — a system

map from static analysis and a golden suite captured mechanically at the boundaries. The methodology's original rule went further and forbade the agent from authoring any specification of what the old system does: an agent-written specification encodes the same partial reading of the code that loses behavior silently, which is precisely the blind spot the golden exists to counter. That prohibition was later renegotiated into a quarantine: the agent may author *candidate* behavioral claims, each carrying typed evidence (`test`, `code`, or `inference`) with machine-resolved references confined to the repository tree and a bounded confidence (inference is always low) — but no candidate reaches the forward flow without a recorded human verdict, `preserve`, `fix`, or `undefined`, in an append-only behavior ledger verified against version control. The gate is measured, not promised: a candidate with no verdict blocks the pull-request state, naming itself. The three verdicts matter because legacy code mixes intent with accident — bugs that customers depend on, timeouts nobody decided — and a naive extraction fossilizes the bugs as features. A `fix` verdict declares its expected divergence, which the golden oracle then checks (a declared divergence that fails to appear reads red), and an advisory round-trip report traces promoted candidates back into the code by identifier. "Never author" became "never promote without judgment": the human stops authoring text and authors verdicts, and writes the migration specification from judged candidates only.

3.2 The development loop

The loop consumes the specification package and proceeds through phases, each guarded. It opens with planning (constitution first; no acceptance criteria, no build) and a coverage gate that triggers boundary characterization tests when the safety net is insufficient. The build itself runs under decision-record discipline: the agent consults the records before touching governed areas and must propose — never silently take — new architectural decisions, while a simplicity gate scores the diff against declared budgets, with test code excluded so that writing tests is never penalized. Verification is a severity-gated review loop that *converges instead of chasing zero* (findings below the declared gate are deferred, not fixed forever), followed by an integration pass across repositories, late test-writing against stabilized code, and an optional rebuild test measuring specification completeness. A pull request opens only when the derived workflow state says the facts allow it. The loop stops at merge: merging is human.

An escalation contract enumerates the situations in which the agent must stop and ask — iteration cap, oscillating findings, a previously-passing test now failing non-trivially, contradictory tool directives, decisions of architectural weight, and any constitution breach. Escalations and their closures are recorded events; an open escalation caps the readiness score until a human resolves it, and resolving a blocker requires an *escape analysis*: which gate or test should have caught this, and what was done about it.

3.3 A worked example in ten steps

Figure 3 walks a deliberately small feature — a discount calculation — through the whole methodology, because the small case makes the failure modes legible. Six of the ten steps are failures or refinements, and that is the point: each deviation is caught by a specific gate rather than by hope. Two advisory passes tighten the specification before any code exists (a vague criterion becomes a testable oracle; a missing out-of-scope section is added). During the build, mutation testing exposes a test that runs but asserts nothing (*coverage was lying*); the gate-integrity check catches and blocks the agent lowering a threshold to pass; the rebuild test diverges on an implicit input case, and the response is to amend the specification, never the code; the simplicity gate cuts a speculative abstraction. Only then does the readiness score — capped by facts, dominated by measured acceptance — reach the human gate, where a person reads the diff and approves the merge.

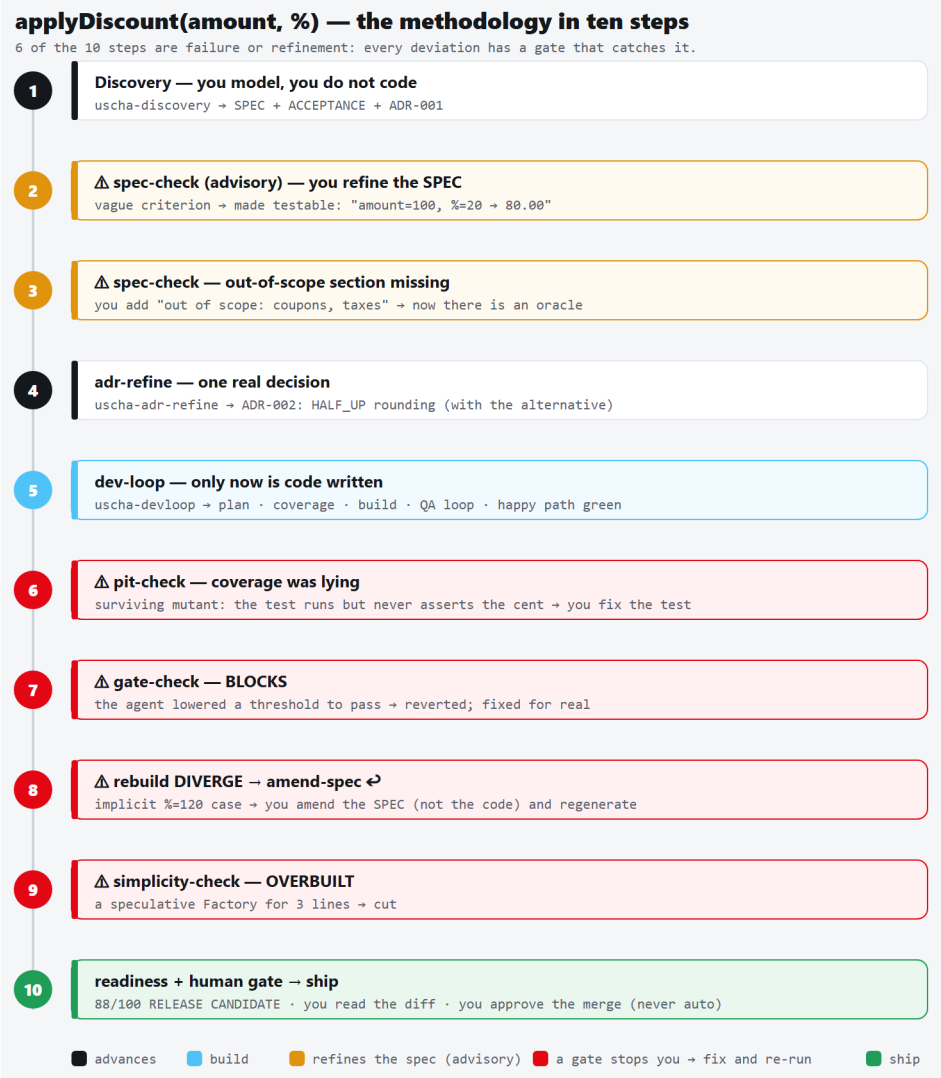


Figure 3: A worked example: `applyDiscount(amount, %)` through the ten steps of the methodology. Black steps advance (blue marks the build), amber steps refine the specification (advisory), red steps are gates stopping the agent (fix and re-run), and the final green step is the human gate. Six of ten steps being failure or refinement is the expected shape of the process, not an anomaly.

4 The Evidence Engine

The engine is a single dependency-free Python file. It never runs an LLM. For tests it runs only the command the project declared (`test-run`) and honors only the reports that run wrote; the other tools it scores (linters, smoke runs, CI references) are still *ingested* by path (JUnit-family XML, Cobertura and lcov coverage, thirteen linter formats across eleven language stacks), and it validates structures. Every verdict is persisted in a deterministic ledger with an integrity checksum; external mutation or a truncated write blocks loading with a recovery message. Table 1 summarizes the complete check family and the verdict class of each.

Fact gates (block). The blocking gates read artifacts, not opinions. Two design choices deserve emphasis. First, the golden gate's masking of volatile fields (timestamps, request identifiers) is itself governed: masking rules are declared in a versioned file that the human approves together with the fixtures, every masked match is reported separately, and any later edit to the rules file is flagged — masking is never invisible. Second, the workflow state machine is *derived*:

Check	Reads	Verdict
`golden-diff`	byte comparison against human-approved fixtures; declared volatiles masked *visibly*	blocks
`gate-check`	diff structure: deleted/disabled tests (nine stack conventions), lowered thresholds, added secrets	blocks
`pit-check`	mutation-testing reports (test effectiveness, because coverage lies)	blocks
`simplicity-check`	diff size, indentation-depth proxy, net growth against declared budgets; test code excluded	advises*
`spec-check` (structure)	missing sections, zero traceable acceptance identifiers, duplicates, invalid rubric structure	blocks
`phase -require pr-ready`	workflow state derived from ledger facts; spike branches always refused	blocks
`readiness`	weighted state score with hard caps and threshold provenance	reports
`regression-check`	findings closed without new non-blank test lines → narrated closure	advises*
`rubric-ingest`	weighted qualitative criteria; evidence-or-nothing grader contract	advises*
`waste-check`	Type-1/2 clone windows of the diff vs the repository — the duplication a diff-local gate cannot see	advises*
`spec-check` (prose)	vagueness and shape heuristics over specification text	advises*

Table 1: The check family. Verdicts marked * advise by default and gate only under explicit human declaration (configuration or flag); the declaration is surfaced everywhere as *requirement (declared)* versus *default (kit opinion)*.

the state (plan, build, qa, escalated, pr-ready) is computed from ledger facts. A self-declared state machine would be narrated state — the exact category of signal the methodology distrusts — so there is nothing to declare and no illegal transition to police.

Witnessed evidence. Since kit 2.10.0 the engine no longer ingests test reports by path alone: `test-run` executes the project's declared test command, hashes the report set before and after, and records the run; acceptance, the workflow state and the seal honor only a report born of a recorded, still-current run. Since 2.11.0 a QA review is witnessed when it starts: the ledger records the tree the review began on, convergence requires each tool's latest review to have seen the source now on disk, and a gated finding closes only through measured movement or, since 2.14.0, a human-confirmed deferral of its id. The design of the latter was model-checked (TLA+/TLC, bounded to three code versions, one tool, one repo), and the check broke the first draft. Provenance proves that a command ran at a tree and wrote bytes, not that it tests anything; the engine does not run the reviewer and cannot prove a review took place.

Declared human acts. Since 2.12.0 the commands that lift the escalated phase require a declared name and reason, record them, and refuse when the process carries a known agent-shell marker; re-initialising a ledger over an open human gate is refused too (ADR-058). Since 2.14.0 confirming the deferral of a gated finding is one of these acts: the finding is deferred by its id, and only a recorded confirmation closes its debt (ADR-060).

Model-checked designs. Since 2.15.0 (ADR-061) a project may name an ADR from which every design records whether it needs a TLA+ model. The engine validates those declarations and each model's manifest — a vacuity configuration that must break, and the red test case each counterexample replays — and runs TLC from a pinned jar, recording the verdict against the hashes of the module, its configuration and its manifest entry. Without Java the models read unmeasured, never passed; it advises unless the project declares a gate. Two of the kit's own designs were checked this way: one model found three holes the code and a fresh reviewer had passed, and the other missed a hole a fresh review then found, which became a configuration

Dimension	Weight	Measured from
acceptance (measured)	30	criteria closed by green, name-tagged test cases in reports born of a recorded, still-current `test-run` (else UNMEASURED)
static gate	20	latest linter runs, normalized severities; never-ran scores 0 (silence is not success)
`adr` (narrative completion)	15	checkbox ratio of the acceptance file — narrated progress, deliberately demoted below measured acceptance
coverage	15	coverage reports against the declared threshold
convergence	10	clean last cycle of every review tool — each tool's latest review witnessed (`review-start`) and anchored to the tree on disk — plus clean persisted fact gates
integration	10	cross-repository contract pass (weight redistributed if disabled)

Table 2: Readiness dimensions and default weights. Hard caps override the average: red tests $\leq$35, open blocker/critical findings $\leq$65, unresolved escalation $\leq$75 — each cap reporting whether its threshold is a declared requirement or a kit default.

that must break.

Structured guesses (advise). The advisory layer includes regression capture (closing findings without adding a single non-blank test line is flagged as *narrated closure*; the failing test goes before the fix); plateau and stop-signal advisories over the finding history (findings flat or rising across three complete cycles means more iteration is not approaching the solution — return to the decision records; everything converged with zero blocking facts means cut and ship); and the rubric layer: a versioned file of weighted qualitative criteria — conventions, error-handling sanity, interface ergonomics, documentation quality — with anchor examples, negative criteria, and a threshold, graded by any runner against a JSON contract with evidence-or-nothing semantics: a verdict that affects the score without a file-and-line citation does not count, duplicate verdicts for one criterion break the contract, and unevaluated criteria count as failed (unevaluated is not approved). The advisory layer also carries *reuse detection*: a deterministic Type-1/Type-2 clone check of the diff against the existing repository. This targets the duplication that a diff-local simplicity gate is blind to by construction — the form of waste most characteristic of generated code [14, 15]. It too reports a fact (a byte-identical window already exists at `file:line`) while leaving the verdict "wasteful" advisory, because a normalized-line proxy has honest false positives (boilerplate, data-transfer objects, embedded SQL) and a check that blocks on those would be disabled and die.

Readiness and the anti-ceremony invariant. A weighted 0–100 score reports the state of the result, never effort (Table 2). The dominant dimension is measured acceptance — criteria closed by green, name-tagged tests from a recorded `test-run` (else UNMEASURED) — with checkbox completion demoted to a narrative dimension. Hard caps override the weighted average, and every biting threshold states its provenance. The score reports; it does not gate — the gates are the facts above. As the gate set grows, the dominant risk shifts from any one bad gate to the *sum* of good ones making the loop feel like an audit — over-processing, the ceremony form of waste. The methodology answers this reflexively with a meta-invariant every future gate must satisfy (it runs without human input, speaks only when it matters, collapses into readiness, and a trivial change skips it), mechanized as a single-verdict view: readiness collapses every persisted gate into one line, expandable on demand. This is the same Lean lens the method applies to code, turned on the method itself [14].

Skill	Front / phase	Role
`discovery`	greenfield front	idea → spec package, one question at a time; the agent *proposes* the system shape
`adr-refine`	known-feature front	Socratic interview → SPEC + decision records + acceptance for an already-scoped feature
`reverse-discovery`	brownfield front	extract *facts* (system map + boundary golden) + candidate claims in quarantine; never promotes without a human verdict
`characterize`	pre-change	capture the approval suite of current behavior — the one artifact the agent must not author
`devloop`	the loop	plan → build → severity-gated review loop → derived PR gate; records every step in the ledger
`sysdoc`	reporting	two-audience (commercial + technical) HTML deck generated from the ledger
`rubric`	advisory gate	thin adapter that grades against a versioned `RUBRIC.md` via the vendor-neutral JSON contract
`mirador`	reporting	read-only dashboard rendered from the ledger: readiness, measured acceptance, loop burn-down, and per-figure receipts
`status`	reporting	the same measured facts as a one-line in-chat readout, for surfaces with no persistent status line

Table 3: The nine agent skills. Each is a markdown instruction file readable by any instruction-following agent; the engine is invoked identically from each.

5 Reference Implementation

The reference implementation packages the methodology as nine agent skills — the two specification fronts (`discovery`, `reverse-discovery`), a Socratic interview for an already-scoped feature (`adr-refine`), the loop orchestrator (`devloop`), golden capture (`characterize`), a two-audience documentation generator (`sysdoc`), the rubric grader adapter (`rubric`), and two read-only status views, a dashboard (`mirador`) and an in-chat readout (`status`) — plus the engine (63 subcommands). Three properties matter for portability. First, the skills are markdown instruction files readable by any instruction-following agent; the engine is invoked identically from any of them. Second, every LLM-judgment interface is a contract: the rubric grader ships as a neutral prompt usable by any vendor's agent, a raw API call, or a human filling the JSON by hand — the implementation's test suite exercises exactly that path, with no LLM in the loop. Third, the mechanical prohibition (the agent must not write approved golden fixtures) is enforced by a pre-tool-use hook at the runtime boundary, not by instruction — during development of the implementation itself the hook blocked one of its author's own commands, which is the property working as designed. Since 2.12.0 a second hook on the same runtime keeps a pull request a draft while the gate is red. Table 3 maps the nine skills to the phase each serves.

The implementation carries its process in executable form. A smoke suite of 457 checks (at kit 2.2.0) exercises the implementation — the engine against synthetic ledgers, plus the installer and the distribution router — including, deliberately, the agnosticism claim: the rubric-layer tests fill the grader's JSON by hand, with no model in the loop. The same run measures the implementation's own acceptance criteria the way the methodology prescribes for any project: 329 of 330 closed on green, name-tagged test evidence at that version. The engine also reports a passive Lean process metric — first-time yield, the fraction of repositories that cleared QA

on the first cycle with no rework or escalation — derived from ledger facts and kept strictly informational, never a gate: it measures the process, not the state of the result. Version consistency across the six release artifacts (version file, configuration, two plugin manifests, marketplace manifest and package manifest, plus the matching changelog) is itself a suite check, converting a release convention into a verifiable fact. An installation-diagnosis subcommand in the spirit of `flutter doctor` verifies interpreter, per-stack toolchains, hook presence and registration, skill integrity, and per-project configuration — and every failing check carries its remedy (an installation link or command), so the workflow is: run, install what is listed, re-run until green. Three installation modes are supported: per-project, per-user, and as a packaged plugin for one agent runtime — packaging, not dependency; the other runtimes install by copying files.

6 Design Validation and Self-Application

No controlled external evaluation has been conducted yet; we report the design-validation evidence honestly and completely.

Adversarial audits. Before stabilization, the methodology and its implementation were subjected to two adversarial audits totaling 231 agent evaluations: a seven-lens audit (54 agents; findings attacked by skeptics defaulting to *refuted*) confirmed 33 weaknesses and refuted 13, with the central finding that the core principle was *inverted in the code* — fact gates existed but were not wired to block. A second audit verified 171 claims across documentation, code, and design intent. The response re-wired every fact gate into the engine and re-passed all documentation against the implemented reality (a recorded truth-pass over the affected documents), converting the principle from slogan to enforced property.

Doctrine cross-examination. The methodology was systematically read against classical software-engineering doctrine [4], producing a set of validations, tensions with recorded resolutions, and ten actionable improvements that shipped as separate releases with their own regression checks (the per-item tally lives in the release changelogs rather than in a single committed artifact, and is stated here at that level of evidence). The validations were often word-for-word: "don't assume it — prove it" is the ledger's two-tier contract; shared mutable resources including *files* motivated the ledger's integrity checksum; and the doctrine's warning against trusting enforcement to good intentions is exactly why the golden prohibition is a hook rather than an instruction. In the two deepest tensions the methodology deliberately departed from the letter of the doctrine to defend its own principle: readiness caps continue to bite by default (their existence is a definition; only the numbers are opinion, and they now state their provenance), and the workflow state machine is derived rather than declared.

Self-application. The implementation is developed under its own process. Across the self-application arc (versions 1.10 through 2.2), each engine change carried smoke checks in the same commit, a fresh-context review before merge, and a version consistency check across every release artifact, enforced by the suite. These reviews repeatedly caught real defects prior to merge — the changelogs record the resulting hardening across the series — including: test-file classifier gaps that made a documented guarantee an overclaim; a blank-line loophole in regression-check evidence (any non-blank test line counts as evidence — a tripwire, not a judge — but a blank line must not); a review-suite check that passed for the wrong reason (its precondition never held, so the assertion was vacuous); silent last-wins semantics on duplicated grader verdicts — a grader gaming vector, now a broken-contract error; and, in the single-verdict release itself, documentation that under-enumerated what the collapsed view actually shows. We take the consistency of this defect stream as evidence for the methodology's core premise: independent, fresh-context checking against recorded criteria catches what the authoring context cannot see.

The Diamond program (versions 1.72–1.99): a bounded controlled evaluation. An *archetype*, in this program, is a system specified completely enough to be built from its

specification alone — a token-bucket rate limiter, a request router, a two-module ledger — and small enough that a withheld oracle can decide whether a given build is the same system. Between July and August 2026 the methodology's central claim — that a canonical specification package is the source and code is a build artifact — was put under its own instruments. A *Diamond Bench* of twelve such archetypes (eleven in Python, one of them a two-module system with a decided seam, and one in JavaScript) was compiled *blind* — the compiling model never sees the oracle, and its working directory holds nothing but what the prompt puts there — from the canonical package alone, and certified by a *withheld oracle* authored before any compilation and shown discriminating against a degenerate stub and per-rule wrong implementations.

The first arm ran three compilers of a single vendor (Claude Haiku, Sonnet and Opus): nine of the twelve archetypes reached PASS and three were PARTIAL, and the oracle caught a convergent aliasing defect two compilers shared. That arm left open the question a reader is entitled to ask — is regeneration fidelity a property of the *method*, or of one model family? — and version 1.99.0 answered it the only way a single arm can, as a falsification test the claim survived (ADR-042). A fourth compiler, from a second vendor (the OpenAI Codex command-line interface driving `gpt-5.5`), compiled all twelve archetypes blind under the same protocol and the same withheld oracles, plus second runs of the ten archetypes that have one: 22 dispatches, all 22 accepted by the engine's own compilation validator, none re-dispatched to obtain a greener result. Two mechanical deviations are recorded rather than smoothed over. The second vendor's tool carries a large system prompt of its own that the first arm's compilers never had. And because the machine's administrator policy forbids that tool's exec-mode file writes, the model returned its source inside the validated JSON and the harness wrote the bytes — defeating an administrator's security control to make a benchmark run is not a method. The second deviation removes nothing the bench measures: the number of shell commands executed is 0 across all 22 dispatches, so neither arm ever ran its own code.

The measured result is 8 PASS, 4 PARTIAL: 8 of 12 archetypes regenerate the system under the withheld oracle, where the single-vendor arm had read nine and three. Exactly one verdict moved, and the move is the point. On `transformer` the canonical specification says each input record has "exactly" three named fields and then never lists an extra field among the error cases; the withheld oracle resolved that silence one way, the new compiler resolved it the other, *declared that reading in its own unresolved-intent record before the oracle judged it*, and reproduced it in a second blind run. The defect is an under-specification in the canonical package that three related models had resolved identically — exactly the blind spot a single-vendor bench cannot see — and it corroborates, from the opposite direction, what the controlled-language arm had recorded about the same sentence sixteen releases earlier.

A *controlled-language arm* compared free-prose authoring of the same package against EARS+STE authoring — the Easy Approach to Requirements Syntax, plus Simplified Technical English — under the same oracle, deconfounded by same-generation runs of both arms, across five archetypes: REDUCED in one, IMPROVED in one, NO EFFECT in two, WORSE in one — dense archetypes helped behaviourally, simple ones did not, and one rewrite sharpened a latent ambiguity into an over-commitment the oracle rejected. The single REDUCED entry (the guard) carries its own qualifiers: it is classified NOISY at an intra/inter ratio of 0.642, and the same comparison reads MIXED under Python 3.8, so by the program's own doctrine structural variance alone no longer carries a claim.

A *noise-floor* run — second blind runs of the same models, over the ten of twelve archetypes that have one (`ledger-lite` and `rate-limiter` have none) and at $n = 2$ per model, the minimum that yields a floor at all, and a floor rather than a distribution — reads NOISY overall: SIGNAL in one entry, NOISY in seven, and outright NOISE in two (`scheduler` 1.20, `worker` 1.10), for a mean intra/inter distance ratio of 0.815, while behaviour is stable (36 of 40 reruns fail the same oracle cases). The cross-vendor arm moved two entries out of NOISE (`parser` 1.13 to 0.73, `state-machine` 1.12 to 0.81) for a mechanical reason worth stating: a

compiler that is not a relative of the other three widens the between-compiler spread more than it raises the same-model floor. Both land within about 0.3 of the class threshold, so the classes are pinned by the suite and the ratios are not. The program's variance claims therefore carry a per-entry qualifier, its controlled-language conclusion is stated behaviour-first, and one earlier variance narrative was retracted.

A *round-trip* instrument measures how much of the human-authored intermediate representation the mechanical reverse organs can anchor in each compiled artifact, counting static and behavioural footing: mean recoverability 0.815, the same value under Python 3.8 and 3.13. The aggregate is published as a range rather than a float because, before the fourth compiler joined, the per-entry means summed to a value whose twelfth part fell exactly on a rounding boundary and printed 0.828 under one interpreter against 0.827 under the other. The instrument's first release read 0.062, with the behavioural dimension unmeasured in every entry, because no oracle case in the bench was tagged with the acceptance criterion it certifies. Closing that gap was a curation, not a re-measurement: a human tagged the twelve withheld oracles case by case — 201 of the 216 cases tagged, the remaining 15 owned by no single criterion, three tags adjusted by hand — leaving payloads and expectations untouched. What changed is what the instrument can attribute; what the compiled artifacts do did not move. The reverse half of the diamond now anchors names *and* behaviour.

Every one of these numbers is regenerated by the engine from committed fixtures and pinned by the suite; most releases passed an independent blind review before they shipped, with two exceptions that say so in their own changelogs — 1.74.0 and 1.75.0 were reviewed inline by the author, and the independent review later run on 1.74.0 caught two report-honesty defects the inline self-review had missed. Those reviews caught real defects in most releases — each catch is recorded in that release's changelog — including two that changed how a result was stated (a spec found self-contradictory, a false-precision ratio). Human curation of the machine-generated code — one verdict per observation, and no observation promoted without one — is complete as of 2026-09-08: 221 verdicts over all twelve archetypes and all four compilers, 213 `preserve` and 8 `fix`, none left unjudged, the eight `fix` verdicts naming defects the oracles had passed. All 221 were recorded by a single curator (the author), and their coverage per archetype is uneven (43 on the guard, 5 on the state machine). This is a controlled evaluation of the artifact-level claim; it is not the operator-level study below.

The release ritual under its own instruments (versions 1.96–2.2, 2026). After the Diamond program the same discipline was turned on the process that ships the implementation, and each of the changes below began as a defect that process found in itself. *The ritual became a program.* The release procedure had been prose a maintainer re-read; since 1.96.0 it is a script (ADR-041) that performs the steps and refuses, naming which of eight invariants it broke — among them that the version surfaces move together, that the suite runs on the code commit and a non-zero exit is a refusal rather than a note, that the evidence commit carries evidence and nothing else, and that the tag is created after the push, so a red continuous-integration run is a wait and never a publish. The same decision retired a wall clock: whether the project's own ledger is fresh is now decided by version-control ancestry — was the ledger recorded in, or after, the commit that last changed the engine? — which deleted a step whose only purpose had been to keep a timestamp ahead of a commit. Roughly half of the readiness history since August 2026 had been that step, plotting the ritual as if it were the product.

Published claims became a gate. Version 1.97.0 derived the counts a document may claim — version, subcommand count, skill count — from the tree itself, and gated one list of published files against them, rewriting a stale claim in place and failing on one it does not recognise. Version 2.2.0 extended the derivation to the benchmark headline, after the gate was caught missing it: the project's own home page went on claiming nine of the twelve archetypes through nine releases after the cross-vendor arm moved one verdict, because the number sat in one markup element and the noun that gives it meaning in the next, and because no derived fact

existed to compare it against. Both halves of that defect were repaired, and a red probe against the previous engine ships beside the check.

Two shipped defaults were found never to have taken effect. Version 2.0.0 measured that the risk presets — which exist to make a trivial change run one review and a high-risk change run three — had never modulated anything in a repository the implementation itself had set up, because the installer copied the reference configuration, so every default arrived as an explicit declaration and outranked the preset by the precedence rule's own terms. The installer now generates a minimal configuration that declares no knob a preset owns. Version 2.1.0 (ADR-043) found the simplicity gate stopping loops on a budget nobody had adopted: a six-line diff scored OVERBUILT because the gate's indentation proxy read one wrapped call argument as nesting depth nine. That gate now advises unless the project declares a numeric budget *and* asks for gating — this paper's own rule, guesses advise, applied to a check that had quietly crossed into blocking.

Two field retrospectives added evidence classes. Version 2.2.0 carried, from two projects using the method in the field, an origin marker recording *who* decided a specification item, so that an item the agent introduced and an item the human chose are no longer the same sentence in the same file (ADR-044); a corpus gate that scores a command against a body of real inputs (ADR-046); ingestion of a project's own smoke run as measured evidence rather than a prose checklist (ADR-047); and an operability check reading four facts out of the tree — a pipeline running the repository's configured test command, a workflow publishing a release asset, a runbook naming start, configuration, rollback and smoke, and a declared seed command whose script exists (ADR-048).

7 Limitations and Threats to Validity

No operator-level controlled evaluation. The Diamond program (§6) is a controlled evaluation of the artifact-level claim — replaceability under a withheld oracle, with a measured noise floor — but all of it was run by the methodology's author on bounded archetypes; effect sizes on team productivity, defect escape rates, or comparison against the frameworks surveyed in [3] remain unmeasured, and the structural distance the program uses is coarse (LOC, AST size, imports) and interpreter-sensitive, which the instrument now states. **Single-operator bias.** Self-application shares authorship context; although reviews run in fresh contexts, they share the model family and the author's configuration. The same bias reaches the Diamond program's human layer: all 221 curation verdicts over the compiled artifacts were recorded by that single operator, with no second curator and no inter-rater measurement. **Two vendors, and one model of the second.** The cross-vendor arm is a falsification test the replaceability claim survived, not a survey: four blind compilers, two vendors, and exactly one model (`gpt-5.5`) of the second, at $n = 2$ runs. Nothing here measures a vendor, and nothing generalizes to language models at large. **Noise-floor coverage.** The floor that qualifies every variance statement rests on ten of the twelve archetypes at $n = 2$ per model; it is a floor, not a distribution, and two archetypes have no second run at all. **Structural distance on JavaScript.** The distance function is three-dimensional for Python (lines of code, AST size, import Jaccard) but two-dimensional for the JavaScript archetype: no standard-library JavaScript AST is available, so `ast_nodes` reads UNMEASURED and the two arms are not directly comparable on that axis. **LLM non-determinism.** The rubric layer's grades vary across runs; the methodology bounds the blast radius (advisory by default, evidence-or-nothing, human-declared gating) but does not eliminate variance. **Scope.** The loop targets a single operator driving one non-trivial change; multi-operator concurrency is out of scope, and the ledger is single-writer by design. **Heuristic residue.** Some gates carry documented approximations; each is disclosed at the point of use, but disclosure is not elimination. Three are worth naming. Path-based test classification and stale-report windows are approximations of what the engine cannot see directly.

The simplicity gate's `max_nesting` is an *indentation proxy*, not a measure of nesting: it divides the leading indentation of added lines by the configured indent width, so wrapped arguments, JSX and multi-line literals inflate it — which is why the gate advises unless a budget is adopted (§6). And the operability check grades *presence*, not quality: it can see that a runbook names a rollback section and that a workflow runs the configured test command, not that the procedure is correct or the tests meaningful. **Field evidence is instrumented, not yet weighted.** The corpus gate scores a command against real inputs and blocks through its own record, but readiness gains no `field` dimension and no weight from it: adding a weighted dimension moves every existing project's score on upgrade, so it is deferred to a decision record of its own rather than smuggled in beside the instrument that would feed it. **No control for reasoning-first.** Independent evidence [20] found that prompting a model to reason about edge cases with no specification captured most of the apparent benefit on simple tasks; this paper does not isolate a reasoning-only baseline from the specification-driven flow, so the quality claim made here is defensible only for the multi-constraint work the methodology targets, not for simple tasks in general. **Reviewer cost is unmeasured.** The human cost of spec-anchored review — measured elsewhere [20] as a substantial increase in reviewer minutes — is not measured in this work. **Trusted declarations and bounded checks.** The declared test command is trusted: it is a configuration line a reviewer reads, not something the engine can validate. A review is witnessed at its start, not proven to have run. The TLA+/TLC check is bounded (three code versions, one tool, one repository). And the full dev loop on Codex was a pilot with $n = 1$, a tiny greenfield repository and no hooks, so the golden write guard is advisory there (ADR-049). A declared human name is not identity: an agent can unset the marker, other harnesses expose none, and the pull-request hook reads text, so an API call or a browser passes it. The only authenticated human act is the forge's own ready-for-review or merge. A green model shows that TLC found no violation of the model within its bounds, not that the code matches it; the replayed red cases are the only link.

8 Future Work

The central gap is operator-level controlled evaluation. Concretely, we plan a study that assigns matched non-trivial changes to operators *with* and *without* the methodology and measures three outcomes the ledger already instruments: defect-escape rate at human review, rework (first-time yield across cycles), and time-to-merge — against two baselines, ad-hoc agent use and the frameworks surveyed in [3] along their six taxonomy dimensions (with particular attention to the validation and portability axes). The missing pieces are operators other than the author and a controlled task set; the process instrumentation is already in place. Supporting steps: read-only dry runs of all eleven stack adapters against real repositories; a multi-project dogfooding phase; and an evaluation of the rubric layer's inter-run variance under anchored versus unanchored criteria, connecting to the validation self-awareness agenda of [2]. A further direction extends the Lean lens beyond the inner loop: the reuse and anti-ceremony gates address two forms of waste (duplication and over-processing), but the "shine" discipline of 5S — scheduled cleanup that keeps the legacy layer from silently freezing — is a candidate *outer-loop* routine [14], deliberately out of scope here because it acts on the repository over time rather than on a single change under review.

A further direction is suggested by the closed cycle itself. With a forward path (specification to code, generated by the agent under the loop's gates) and a reverse path (code to curated candidate specifications), the specification package plus its behavior ledger begins to resemble the canonical representation of a system, with code as a regenerable build artifact — the asset inverts. This is no longer purely positional: the round-trip we once deferred has since been run (§6) as `bench-roundtrip` over the twelve Diamond Bench archetypes. It measures, for every compiled artifact, how much of the human-authored intermediate representation the mechanical

reverse organs (static extraction, the withheld oracle's own case results) can anchor — deliberately *not* by regenerating a specification from code and diffing it, which would reintroduce the narrated inference the doctrine forbids. The measured result is stated as measured: mean recoverability 0.815 (the aggregate being pinned as a range rather than a float, because an earlier sum fell on a rounding boundary), counting only static and behavioural footing (the compiler's own trace-manifest claims are excluded as tautological, since the blind prompt supplied the very node identifiers being "recovered"). It read 0.062 in the instrument's first release, with the behavioural dimension unmeasured across all twelve entries, because no oracle case in the bench was tagged against the acceptance criterion it certifies — recorded then as an instrumentation gap rather than a null result, together with the concrete next step. That step was taken: the twelve withheld oracles were curated case by case against the criteria, by a human and with the payloads and expectations left untouched, so what changed is what the instrument can attribute and not what the compiled artifacts do. The distinction matters more than the number — an absence that was named is allowed to be closed, and the reader can see which of the two moved. What the round trip shows now is that mechanical reverse discovery anchors *names* and *behaviour*, still without regenerating a specification. The same discipline was applied to the tempting unification "drift detection is incremental reverse discovery": recorded as an equivalence of idea, rejected as shared architecture, because one instrument advises about staleness while the other proposes content.

As models improve, the package's value plausibly shifts: from "regenerates better" toward *auditable contract* — every verdict attributable to a named criterion and, where a human judged, to the person who signed it. That is what the ledger, `bench-curate -human`, and the `origin: agent` markers (§6) together produce, and it is a different value proposition than regeneration fidelity, one that does not erode as models close the fidelity gap.

9 Conclusion

Uscha's contribution is not a new agent architecture but a discipline for placing trust: facts block, guesses advise, the measured overrides the narrated, and the human decides at exactly three points. Applied one level up, that same discipline yields the thesis the reference implementation now instruments: the specification package is the source and code is a build artifact — bounded systems regenerate to the same behavior under a withheld oracle, and how much of the asset survives the round trip is a published number with a published ceiling, not a promise. The methodology's own history — a central principle found inverted by adversarial audit, then wired into an engine that now blocks its own author when he crosses the line — suggests that for LLM-assisted development, the difference between a methodology and a slogan is whether the checks are executable. The reference implementation is released as open source under the MIT license, published as the npm package `@andresmassello/uscha` with its repository at https://github.com/andresmassello/uscha.

References

[1] Z. Ji, Z. Zhang, C. Xu, Z. Li, Y. Gao, S. Wang, and S.-C. Cheung, "Coding Agents Are Guessing: Measuring Action-Boundary Violations in Underspecified DevOps Instructions," arXiv:2607.02294 [cs.SE], 2026.

[2] Y. Ma, B. Kereopa-Yorke, and B. Schultz, "Building to the Test: Coding Agents Deliver What You Check, Not What You Requested," arXiv:2606.28430 [cs.SE], 2026.

[3] S. Oliveira de Macedo, "From Prompt to Process: a Process Taxonomy and Comparative Assessment of Frameworks Supporting AI Software Development Agents," arXiv:2606.04967 [cs.SE], 2026.

[4] D. Thomas and A. Hunt, *The Pragmatic Programmer: Your Journey to Mastery*, 20th Anniversary Edition. Addison-Wesley, 2019.

[5] M. Nygard, "Documenting Architecture Decisions," blog post, 2011. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

[6] B. Böckeler, "Maintainability Sensors for Coding Agents," martinfowler.com, 2025. https://martinfowler.com/articles/sensors-for-coding-agents.html

[7] A. Osmani, "Agentic Code Review" and "Loop Engineering," addyosmani.com, 2024–2025.

[8] J. Maeda, *The Laws of Simplicity*. MIT Press, 2006.

[9] G. Huntley, "The Ralph Wiggum Loop," blog post, 2025.

[10] Anthropic, "Building Effective Agents" (the evaluator-optimizer workflow), 2024. https://www.anthropic.com/research/building-effective-agents

[11] M. Fowler et al., "Exploring Generative AI," memo series, martinfowler.com, 2023–2026. https://martinfowler.com/articles/exploring-gen-ai.html

[12] L. Falco et al., "ApprovalTests," https://approvaltests.com.

[13] M. Strathern, "'Improving ratings': audit in the British University system," *European Review*, vol. 5, no. 3, pp. 305–321, 1997.

[14] M. Poppendieck and T. Poppendieck, *Implementing Lean Software Development: From Concept to Cash*. Addison-Wesley, 2006.

[15] GitClear, "AI Copilot Code Quality / The Maintainability Gap," industry research report, 2024–2026. https://www.gitclear.com

[16] S. Alvanakis Apostolou, J. Bosch, and H. Holmström Olsson, "Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the Software Development Life Cycle," arXiv:2605.15245 [cs.SE], 2026.

[17] V. Tawosi, K. Ramani, S. Alamir, and X. Liu, "ALMAS: an Autonomous LLM-based Multi-Agent Software Engineering Framework," arXiv:2510.03463 [cs.SE], 2025.

[18] D. Polivaev, "spec-loop: a design-first AI-assisted development framework," open-source project, 2026. https://github.com/dpolivaev/spec-loop

[19] W. Lin, "Self-driving codebases," Cursor Research, February 2026. https://cursor.com/blog/self-driving-codebases

[20] N. Garg, "When Spec-Driven Development Pays Off," InfoQ, September 2026. https://www.infoq.com/articles/when-spec-driven-development-pays-off/

[21] O. Pop, "Cloudflare Introduces the Agent Development Lifecycle Stack to Replace Traditional SDLC," InfoQ, 21 September 2026. https://www.infoq.com/news/2026/09/cloudflare-adlc-agents/